

lab3_rag(1)

June 10, 2026

1 Lab 3 — Full RAG Pipeline

1.1 What We Build Today

Medical Book (PDF)
↓
Chunk text (sentence-level)
↓
Embed + Store in Weaviate Cloud
↓
Retrieve using 4 strategies:
- Semantic Search
- BM25 Search
- Hybrid Search
- Semantic + Reranking
↓
Format → Inject into prompt
↓
Groq LLM → Response
↓
Widget - compare all strategies side by side

1.2 1 Setup & Connect

```
[10]: import os
import pymupdf as fitz
import nltk
import numpy as np
import weaviate
import weaviate.classes as wvc
from weaviate.classes.query import Filter, Rerank
import ollama
from groq import Groq
from dotenv import load_dotenv
import ipywidgets as widgets
from IPython.display import display, Markdown, clear_output

nltk.download('punkt',      quiet=True)
```

```

nltk.download('punkt_tab', quiet=True)

load_dotenv()

# Clients
weaviate_client = weaviate.connect_to_weaviate_cloud(
    cluster_url=os.getenv('WEAVIATE_URL'),
    auth_credentials=weaviate.auth.AuthApiKey(os.getenv('WEAVIATE_API_KEY'))
)
groq_client = Groq(api_key=os.getenv('GROQ_API_KEY'))

COLLECTION_NAME = 'MedicalBook'
LLM_MODEL        = 'llama-3.1-8b-instant'
PDF_PATH         = 'data/medical_book.pdf'

print(' Weaviate:', weaviate_client.is_ready())
print(' Groq client ready')

```

```

/home/abdelalim/projects/stages/RAG/medical-rag-
advanced/.venv/lib/python3.11/site-packages/weaviate/warnings.py:312:
ResourceWarning: Con004: The connection to Weaviate was not closed properly.
This can lead to memory leaks.
    Please make sure to close the connection using `client.close()`.
warnings.warn(
/tmp/ipykernel_10302/374370473.py:20: ResourceWarning: unclosed <ssl.SSLSocket
fd=79, family=2, type=1, proto=6, laddr=('192.168.1.11', 33438),
raddr=('34.36.172.118', 443)>
    weaviate_client = weaviate.connect_to_weaviate_cloud(
ResourceWarning: Enable tracemalloc to get the object allocation traceback

Weaviate: True
Groq client ready

```

1.3 2 Embed Function (Ollama — local)

```

[11]: def embed(text: str) -> list[float]:
    """
    Embed text using Ollama nomic-embed-text (local).
    Returns a list of floats.
    """
    response = ollama.embeddings(model='nomic-embed-text', prompt=text)
    return response['embedding']

print(f' Embedding dim: {len(embed("test"))}')

```

Embedding dim: 768

1.4 3 Ingest PDF into Weaviate

Extract → sentence chunk → embed → store in Weaviate.

Run this **once only**. Skip if collection already exists.

```
[12]: collections = weaviate_client.collections.list_all()
for name in collections:
    weaviate_client.collections.delete(name)
    print(f'    Deleted: {name}')

print(' Weaviate is clean - ready to ingest')
```

Weaviate is clean - ready to ingest

```
[13]: from nltk.tokenize import sent_tokenize

def extract_and_chunk(pdf_path: str, sentences_per_chunk: int = 4, overlap: int =
    1) -> list[dict]:
    """Extract PDF and split into sentence-level chunks."""
    doc = fitz.open(pdf_path)
    chunks = []
    chunk_id = 0
    for page_num, page in enumerate(doc):
        text = page.get_text().strip()
        if not text:
            continue
        sentences = sent_tokenize(text)
        i = 0
        while i < len(sentences):
            chunk_text = ' '.join(sentences[i:i + sentences_per_chunk]).strip()
            if chunk_text:
                chunks.append({'chunk_id': chunk_id, 'text': chunk_text, 'page':
    page_num + 1})
            chunk_id += 1
            i += sentences_per_chunk - overlap
        doc.close()
    return chunks

def ingest_to_weaviate(chunks: list[dict]):
    """Delete old collection, create new one and batch insert chunks."""
    # Clean start
    if weaviate_client.collections.exists(COLLECTION_NAME):
        weaviate_client.collections.delete(COLLECTION_NAME)
        print(f'    Deleted: {COLLECTION_NAME}')

    # Create collection with BM25 tokenization enabled
    weaviate_client.collections.create(
```

```

name=COLLECTION_NAME,
vectorizer_config=wvc.config.Configure.Vectorizer.none(),
properties=[
    wvc.config.Property(
        name='text',
        data_type=wvc.config.DataType.TEXT,
        tokenization=wvc.config.Tokenization.WORD    # enables BM25
    ),
    wvc.config.Property(name='page',      data_type=wvc.config.DataType.
↪INT),
    wvc.config.Property(name='chunk_id', data_type=wvc.config.DataType.
↪INT),
]
)
print(f' Collection created: {COLLECTION_NAME}')

# Batch insert
collection = weaviate_client.collections.get(COLLECTION_NAME)
total      = len(chunks)
with collection.batch.dynamic() as batch:
    for i, chunk in enumerate(chunks):
        batch.add_object(
            properties={'text': chunk['text'], 'page': chunk['page'],
↪'chunk_id': chunk['chunk_id']},
            vector=embed(chunk['text']))
        )
    if (i + 1) % 50 == 0 or (i + 1) == total:
        print(f'    {i + 1}/{total} chunks...')

print(f'\n Done - {total} chunks in Weaviate')

chunks = extract_and_chunk(PDF_PATH)
print(f' {len(chunks)} chunks created')
ingest_to_weaviate(chunks)

```

2237 chunks created

```

/home/abdelalim/projects/stages/RAG/medical-rag-
advanced/.venv/lib/python3.11/site-packages/weaviate/warnings.py:196:
DeprecationWarning: Dep024: You are using the `vectorizer_config` argument in
`collection.config.create()`, which is deprecated.
    Use the `vector_config` argument instead.

```

```
warnings.warn(
```

```

Collection created: MedicalBook
50/2237 chunks...

```

100/2237 chunks...
150/2237 chunks...
200/2237 chunks...
250/2237 chunks...
300/2237 chunks...
350/2237 chunks...
400/2237 chunks...
450/2237 chunks...
500/2237 chunks...
550/2237 chunks...
600/2237 chunks...
650/2237 chunks...
700/2237 chunks...
750/2237 chunks...
800/2237 chunks...
850/2237 chunks...
900/2237 chunks...
950/2237 chunks...
1000/2237 chunks...
1050/2237 chunks...
1100/2237 chunks...
1150/2237 chunks...
1200/2237 chunks...
1250/2237 chunks...
1300/2237 chunks...
1350/2237 chunks...
1400/2237 chunks...
1450/2237 chunks...
1500/2237 chunks...
1550/2237 chunks...
1600/2237 chunks...
1650/2237 chunks...
1700/2237 chunks...
1750/2237 chunks...
1800/2237 chunks...
1850/2237 chunks...
1900/2237 chunks...
1950/2237 chunks...
2000/2237 chunks...
2050/2237 chunks...
2100/2237 chunks...
2150/2237 chunks...
2200/2237 chunks...
2237/2237 chunks...

Done - 2237 chunks in Weaviate

1.5 4 Retrieval Functions

We implement 4 retrieval strategies following the same pattern.

```
[22]: def get_collection():
    return weaviate_client.collections.get(COLLECTION_NAME)

def _preview_first_result(label: str, docs: list[dict]):
    if not docs:
        print(f' {label}: (no results)')
        return
    print(f" {label}: {docs[0]['text'][:100]}")

    # Strategy 1: Semantic Search
def semantic_search_retrieve(query: str,
                             collection,
                             top_k: int = 5) -> list[dict]:
    """
    Embed query + find most similar vectors in Weaviate.
    Pure vector similarity search.

    Args:
        query      : search query string
        collection : Weaviate collection object
        top_k      : number of results to return

    Returns:
        list of dicts with 'text' and 'page'
    """
    results = collection.query.near_vector(
        near_vector=embed(query),
        limit=top_k,
        return_metadata=wvc.query.MetadataQuery(distance=True)
    )
    return [{'text': o.properties['text'], 'page': o.properties['page']} for o
    ↪in results.objects]

    # Strategy 2: BM25 Search
def bm25_retrieve(query: str,
                  collection,
                  top_k: int = 5) -> list[dict]:
    """
    Keyword-based BM25 search - no embeddings needed.
    Best for exact term matching.
```

```

Args:
    query      : search query string
    collection : Weaviate collection object
    top_k      : number of results to return

Returns:
    list of dicts with 'text' and 'page'
    """
results = collection.query.bm25(
    query=query,
    limit=top_k
)
return [{'text': o.properties['text'], 'page': o.properties['page']} for o
in results.objects]

# Strategy 3: Hybrid Search
def hybrid_retrieve(query: str,
                   collection,
                   alpha: float = 0.5,
                   top_k: int = 5) -> list[dict]:
    """
    Hybrid search = BM25 + Semantic (RRF fusion).
    alpha=0 -> pure BM25, alpha=1 -> pure semantic.

    Args:
        query      : search query string
        collection : Weaviate collection object
        alpha      : balance between BM25 (0) and semantic (1)
        top_k      : number of results to return

    Returns:
        list of dicts with 'text' and 'page'
        """
    results = collection.query.hybrid(
        query=query,
        vector=embed(query),
        alpha=alpha,
        limit=top_k
    )
    return [{'text': o.properties['text'], 'page': o.properties['page']} for o
in results.objects]

# Strategy 4: Semantic + Reranking
def semantic_search_with_reranking(query: str,
                                   rerank_property: str,

```

```

        collection,
        rerank_query: str = None,
        top_k: int = 5) -> list[dict]:

    """
    Semantic search + reranking for more precise results.
    Uses a local fallback because this Weaviate backend does not expose the
    ↪rerank capability.

    Args:
        query          : initial semantic search query
        rerank_property : property to rerank on (e.g. 'text')
        collection      : Weaviate collection object
        rerank_query    : optional different query for reranking
        top_k           : number of results to return

    Returns:
        list of dicts with 'text' and 'page'
    """
    rerank_q = rerank_query if rerank_query else query
    results = collection.query.near_vector(
        near_vector=embed(query),
        limit=max(top_k, 10),
        return_metadata=wvc.query.MetadataQuery(distance=True)
    )
    docs = [{ 'text': o.properties['text'], 'page': o.properties['page'] } for o in
    ↪results.objects]
    query_terms = {term for term in rerank_q.lower().split() if term}
    scored_docs = []
    for doc in docs:
        text_terms = set(doc.get(rerank_property, doc.get('text', '')).lower().
        ↪split())
        score = len(query_terms & text_terms)
        scored_docs.append((score, doc))
    scored_docs.sort(key=lambda item: item[0], reverse=True)
    return [doc for score, doc in scored_docs[:top_k]]

# Quick test
col = get_collection()
q = 'What diseases affect the skin?'

_preview_first_result('Semantic', semantic_search_retrieve(q, col, top_k=1))
_preview_first_result('BM25', bm25_retrieve(q, col, top_k=1))
_preview_first_result('Hybrid', hybrid_retrieve(q, col, top_k=1))
_preview_first_result('Reranked', semantic_search_with_reranking(q, 'text',
    ↪col, top_k=1))

```



```

Semantic: 138
UE 7. Inflammation - Immunopathologie - Poumon - San
Liquide
mécanique
Liquide inflammatoire
L
BM25: (no results)
Hybrid: 138
UE 7. Inflammation - Immunopathologie - Poumon - San
Liquide
mécanique
Liquide inflammatoire
L
Reranked: 138
UE 7. Inflammation - Immunopathologie - Poumon - San
Liquide
mécanique
Liquide inflammatoire
L

```

1.6 5 Format Documents

```

[16]: DEFAULT_PROMPT = (
    'You are a helpful medical assistant. '
    'Use ONLY the following documents to answer the question accurately.\n\n'
    'Documents:\n{documents}\n\n'
    'Question: {query}\n'
    'Answer:'
)

def format_docs(docs: list[dict]) -> str:
    """Format retrieved chunks into a structured string for the prompt."""
    return '\n\n'.join(
        [f'Document {i} (Page {d["page"]}): \n{d["text"]}' for i, d in
         enumerate(docs, 1)]
    )

print(' format_docs ready')

format_docs ready

```

1.7 6 Generate Final Prompt

```

[17]: def generate_final_prompt(query: str,
                                top_k: int,
                                retrieve_function: callable,
                                rerank_query: str = None,
                                rerank_property: str = None,

```

```

        use_rerank: bool = False,
        use_rag: bool = True) -> str:
    """
    Build the final prompt - optionally with RAG and reranking.

    Args:
        query          : user query
        top_k          : number of chunks to retrieve
        retrieve_function: one of semantic/bm25/hybrid retrieve functions
        rerank_query    : optional reranking query
        rerank_property : property used for reranking
        use_rerank      : whether to apply reranking
        use_rag         : if False, return query as-is (no retrieval)

    Returns:
        str: fully composed prompt
    """
    if not use_rag:
        return query

    col = get_collection()

    if use_rerank and rerank_property:
        docs = semantic_search_with_reranking(
            query=query,
            rerank_property=rerank_property,
            collection=col,
            rerank_query=rerank_query,
            top_k=top_k
        )
    else:
        docs = retrieve_function(query, col, top_k)

    return DEFAULT_PROMPT.format(query=query, documents=format_docs(docs))

print(' generate_final_prompt ready')

generate_final_prompt ready

```

1.8 7 LLM Call via Groq

```

[18]: def llm_call(query: str,
        retrieve_function: callable = None,
        top_k: int = 5,
        use_rag: bool = True,
        use_rerank: bool = False,
        rerank_property: str = None,

```

```

        rerank_query: str = None) -> str:
    """
    Full RAG pipeline - retrieve + format + LLM.

    Args:
        query            : user question
        retrieve_function: retrieval strategy to use
        top_k            : number of chunks
        use_rag          : toggle RAG on/off
        use_rerank       : apply reranking
        rerank_property  : property to rerank on
        rerank_query     : optional rerank query

    Returns:
        str: LLM response content
    """
    prompt = generate_final_prompt(
        query=query,
        top_k=top_k,
        retrieve_function=retrieve_function,
        rerank_query=rerank_query,
        rerank_property=rerank_property,
        use_rerank=use_rerank,
        use_rag=use_rag
    )
    response = groq_client.chat.completions.create(
        model=LLM_MODEL,
        messages=[{'role': 'user', 'content': prompt}],
        max_tokens=500
    )
    return response.choices[0].message.content

# Quick test
query = 'What diseases affect the integumentary system?'
print(' Semantic RAG:')
print(llm_call(query, retrieve_function=semantic_search_retrieve, use_rag=True))
print('\n No RAG:')
print(llm_call(query, use_rag=False))

```

Semantic RAG:

Based on the provided documents, the answer would be related to the conditions mentioned under the "Connectivite" section in Document 2 (Page 138).

The conditions that affect the integumentary system are:

- Phénomène de Raynaud (Raynaud's phenomenon) in Sclérodermie-CREST syndrome
- Sclérodactylie is a characteristic feature of Sclérodermie-CREST syndrome
- Dermato) polymyosites

- Syndrome de Gougerot-Sjögren, which mentions parotidites (parotitis is inflammation of the parotid gland)

No RAG:

The integumentary system, which includes the skin, hair, nails, and associated glands, can be susceptible to various diseases. Here are some examples:

1. **Skin Cancer**: Basal cell carcinoma, squamous cell carcinoma, and melanoma are the most common types of skin cancer.
2. **Acne**: A chronic inflammatory skin condition characterized by comedones (blackheads and whiteheads), pimples, and cysts.
3. **Psoriasis**: An autoimmune disorder that causes red, scaly patches on the skin, often accompanied by joint pain and inflammation.
4. **Eczema (Atopic Dermatitis)**: A chronic skin condition characterized by itchy, inflamed, and dry skin.
5. **Rosacea**: A chronic skin condition that causes redness, flushing, and acne-like symptoms.
6. **Impetigo**: A highly contagious bacterial skin infection that causes red sores and crusting.
7. **Ringworm**: A fungal infection that causes circular patches of itching, red, and inflamed skin.
8. **Skin Infections**: Bacterial infections such as MRSA (methicillin-resistant Staphylococcus aureus), fungal infections (e.g., athlete's foot), and viral infections (e.g., hand-foot-and-mouth disease).
9. **Burns**: Thermal, chemical, or electrical burns that can cause damage to the skin and underlying tissues.
10. **Erysipelas**: A bacterial skin infection that causes a red, warm, and swollen patch on the skin.
11. **Lichen planus**: A chronic inflammatory skin condition that causes itching, blisters, and rashes.
12. **Dermatitis herpetiformis**: A skin condition characterized by blistering and itching, often associated with celiac disease.
13. **Hives (Urticaria)**: An allergic reaction that causes red, itchy, and swollen patches on the skin.
14. **Seborrheic keratoses**: Benign growths on the skin that can become infected.
15. **Fungal skin infections**: Such as thrush, onychomycosis (fungal nail infection), or candidiasis (yeast infection).
16. **Lichen sclerosus**: A chronic skin condition that causes thinning and whitening of the skin, often accompanied by itching and discomfort.
17. **Pityriasis alba**: A skin condition characterized by mild, scaly

1.9 8 Interactive Widget

Compare **5 strategies** side by side: 1. Semantic Search 2. Semantic + Reranking 3. BM25 4. Hybrid 5. No RAG

```
[23]: def display_widget(llm_call_func, semantic_fn, bm25_fn, hybrid_fn, rerank_fn):

    # Inputs
    query_input = widgets.Text(
        placeholder='e.g. What diseases affect the skin?',
        description='Query:',
        layout=widgets.Layout(width='100%')
    )
    rerank_input = widgets.Text(
        placeholder='e.g. text',
        description='Rerank prop:',
        value='text',
        layout=widgets.Layout(width='50%')
    )
    topk_slider = widgets.IntSlider(
        value=5, min=1, max=15, step=1,
        description='Top K:',
        style={'description_width': 'initial'}
    )
    run_btn = widgets.Button(
        description=' Run All',
        button_style='primary',
        layout=widgets.Layout(width='150px')
    )
    status = widgets.Output()

    # Output boxes
    box_style = {'border': '1px solid #ccc', 'padding': '10px',
                 'min_height': '150px', 'overflow': 'auto'}
    out1 = widgets.Output(layout=widgets.Layout(**box_style))
    out2 = widgets.Output(layout=widgets.Layout(**box_style))
    out3 = widgets.Output(layout=widgets.Layout(**box_style))
    out4 = widgets.Output(layout=widgets.Layout(**box_style))
    out5 = widgets.Output(layout=widgets.Layout(**box_style))

    labels = [
        widgets.HTML('<b style="color:#2196F3">1. Semantic</b>'),
        widgets.HTML('<b style="color:#9C27B0">2. Semantic + Rerank</b>'),
        widgets.HTML('<b style="color:#FF9800">3. BM25</b>'),
        widgets.HTML('<b style="color:#4CAF50">4. Hybrid</b>'),
        widgets.HTML('<b style="color:#F44336">5. No RAG</b>'),
    ]
    outputs = [out1, out2, out3, out4, out5]

    def on_run(b):
        for o in outputs:
            o.clear_output()
```

```

status.clear_output()

query    = query_input.value.strip()
top_k    = topk_slider.value
rerank_p = rerank_input.value.strip() or 'text'

if not query:
    with status: print('  Enter a query.')
    return

with status: print('  Generating...')

configs = [
    dict(retrieve_function=semantic_fn, use_rag=True,
↪use_rerank=False),
    dict(retrieve_function=semantic_fn, use_rag=True,
↪use_rerank=True, rerank_property=rerank_p),
    dict(retrieve_function=bm25_fn, use_rag=True,
↪use_rerank=False),
    dict(retrieve_function=hybrid_fn, use_rag=True,
↪use_rerank=False),
    dict(retrieve_function=None, use_rag=False,
↪use_rerank=False),
]

for out, cfg in zip(outputs, configs):
    with out:
        resp = llm_call_func(query=query, top_k=top_k, **cfg)
        display(Markdown(resp))

status.clear_output()

run_btn.on_click(on_run)

# Layout
display(
    widgets.HTML('<h3> RAG Comparison Widget</h3>'),
    query_input,
    widgets.HBox([topk_slider, rerank_input]),
    run_btn,
    status,
    widgets.HBox(labels, layout=widgets.
↪Layout(justify_content='space-around')),
    widgets.HBox(outputs, layout=widgets.
↪Layout(justify_content='space-between'))
)

```

```

display_widget(llm_call, semantic_search_retrieve, bm25_retrieve,
↳hybrid_retrieve, semantic_search_with_reranking)

HTML(value='<h3> RAG Comparison Widget</h3>')

Text(value='', description='Query:', layout=Layout(width='100%'), placeholder='e.
↳g. What diseases affect the s...

HBox(children=(IntSlider(value=5, description='Top K:', max=15, min=1,
↳style=SliderStyle(description_width='in...

Button(button_style='primary', description=' Run All',
↳layout=Layout(width='150px'), style=ButtonStyle())

Output()

HBox(children=(HTML(value='<b style="color:#2196F3">1. Semantic</b>'),
↳HTML(value='<b style="color:#9C27B0">2....

HBox(children=(Output(layout=Layout(border_bottom='1px solid #ccc',
↳border_left='1px solid #ccc', border_right...

```

1.10 9 Close Connection

```

[24]: weaviate_client.close()
print(' Done')

```

Done

1.11 Lab 3 Summary

Step	What we did
Ingest	PDF → sentence chunks → embed → Weaviate
Semantic	Vector similarity search
BM25	Keyword search
Hybrid	BM25 + Semantic (RRF)
Reranking	Semantic + reranker for precision
Widget	5-way side-by-side comparison

1.11.1 Stack

PDF → PyMuPDF
 Chunking → Sentence-level (NLTK)
 Embeddings → Ollama nomic-embed-text (local)
 Vector DB → Weaviate Cloud
 LLM → Groq llama-3.1-8b-instant